

Подготовка к экзамену по C++ (ИТМО)

Тема 15: RAII и Умные указатели (Расширенная теория + Практика)

Hardcore Theory Edition

14 июня 2026 г.

1 Идеология RAII (Resource Acquisition Is Initialization)

1.1 Теория и проблематика

В языках без сборщика мусора разработчик отвечает за жизненный цикл ресурсов (память, файлы, мьютексы, сетевые соединения). Ручное управление через `new/delete` или `fopen/fclose` уязвимо к человеческим ошибкам и, что важнее, к **исключениям**.

Если между захватом ресурса и его освобождением выбрасывается исключение, выполнение функции прерывается. Происходит **Stack Unwinding (раскрутка стека)**: программа выходит из функций, уничтожая все локальные объекты на стеке, пока не найдет `catch`. Если ресурс управлялся сырым указателем, функция очистки никогда не вызовется — произойдет утечка.

Паттерн RAII: 1. Ресурс захватывается в конструкторе объекта. 2. Ресурс освобождается в деструкторе. 3. Объект создается на стеке. Так как при раскрутке стека деструкторы вызываются *гарантированно*, ресурс в безопасности.

Суть на пальцах (Жизненная аналогия)

Представь, что ты взял в библиотеке редкую книгу (выделил память). Если ты управляешь всем вручную, ты должен сам помнить, что книгу надо вернуть. А если по пути домой тебя сбила машина (вылетело исключение), книга навсегда потеряна для библиотеки (утечка памяти).

RAII — это когда при выдаче книги на тебя надевают магнитный браслет. Как только ты пытаешься выйти за двери библиотеки (конец области видимости), рамка безопасности автоматически вырывает книгу из твоих рук и кладет на полку (вызывается деструктор).

1.2 Пример использования

```
1 void bad_function() {
2     int* ptr = new int[1000];
3     if (some_condition) {
4         // Утечка памяти! delete не вызовется
5         throw std::runtime_error("Error!");
6     }
7     delete[] ptr;
8 }
```

```
9
10 void good_function() {
11     // std::vector внутри себя реализует RAII
12     std::vector<int> vec(1000);
13     if (some_condition) {
14         throw std::runtime_error("Error!");
15     }
16     // При throw деструктор vec вызовется автоматически и очистит память
17 }
```

2 std::unique_ptr (Эксклюзивное владение)

2.1 Глубокая теория

`unique_ptr` — это легковесная обертка над сырым указателем.

- **Семантика:** Владеет объектом *единолично*. Два `unique_ptr` не могут указывать на один объект. Копирование запрещено (= `delete`), доступно только перемещение (`std::move`).
- **Zero Overhead:** Размер `unique_ptr` со стандартным удалителем в точности равен размеру сырого указателя (8 байт). Нет никаких дополнительных аллокаций.
- **Empty Base Optimization (ЕВО):** `unique_ptr` позволяет задать кастомный deleter (например, для закрытия файла через `fclose`). Deleter передается как параметр шаблона. Если deleter — это объект без полей (stateless functor), `unique_ptr` приватно наследуется от него. Благодаря ЕВО размер умного указателя не увеличивается, оставаясь равным 8 байтам.

Суть на пальцах (Жизненная аналогия)

`unique_ptr` — это физический ключ от банковской ячейки, существующий в единственном экземпляре. Ты не можешь сделать дубликат ключа (копирование запрещено). Если твой друг хочет открыть ячейку, ты должен отдать ему свой ключ целиком (перемещение `std::move`). Теперь друг владеет ячейкой, а у тебя пустые руки (твой указатель стал `nullptr`).

2.2 Пример использования

```
1 #include <memory>
2 #include <iostream>
3
4 struct MyClass {
5     MyClass() { std::cout << "Created\n"; }
6     ~MyClass() { std::cout << "Destroyed\n"; }
7     void do_something() {}
8 };
9
10 // Функция забирает владение объектом
11 void take_ownership(std::unique_ptr<MyClass> ptr) {
12     ptr->do_something();
13 } // Здесь ptr уничтожается
14
15 int main() {
16     // Создание (C++14 и выше, предпочтительный способ)
17     std::unique_ptr<MyClass> ptr1 = std::make_unique<MyClass>();
18
19     // ОШИБКА КОМПИЛЯЦИИ! Копирование запрещено.
20     // Нельзя написать: auto ptr2 = ptr1;
21
22     // Передаем владение: ptr1 обнуляется, функция забирает объект
23     take_ownership(std::move(ptr1));
24
25     if (!ptr1) {
26         std::cout << "ptr1 is now empty.\n";
27     }
```

```
28
29 // Кастомный Deleter
30 auto file_closer = [](FILE* f) { if(f) fclose(f); };
31 std::unique_ptr<FILE, decltype(file_closer)> file_ptr(fopen("test.txt", "r"), file_closer);
32
33 // Файл закроется автоматически при выходе из области видимости
34 return 0;
35 }
```

3 std::shared_ptr (Совместное владение)

3.1 Глубокая теория

Реализует концепцию подсчета ссылок (Reference Counting).

- **Архитектура:** Размер — 16 байт (2 указателя). Хранит указатель на объект и указатель на **Control Block** (управляющий блок в куче).
- **Control Block:** Содержит `shared_count` (счетчик владельцев), `weak_count` (счетчик наблюдателей) и сам кастомный `deleter`. В отличие от `unique_ptr`, тип `deleter`'а стирается (Type Erasure) и не является частью типа `shared_ptr`.
- **Проблема двух аллокаций:** Конструкция `std::shared_ptr<int> p(new int(5));` работает в два этапа. Сначала `new int` выделяет память под число. Затем конструктор `shared_ptr` выделяет память под Control Block. Это медленно и вызывает фрагментацию кэша.
- **Решение (make_shared):** Функция `std::make_shared` выделяет **один сплошной кусок памяти**, в котором лежит и сам объект, и Control Block. Это быстрее и эффективнее.

Суть на пальцах (Жизненная аналогия)

`shared_ptr` — это совместный доступ к Google Документу. У каждого есть ссылка. Google (Control Block) ведет счетчик: "Документ открыт у 5 человек". Вы скидываете ссылку друзьям (копирование, счетчик растет). Когда студент закрывает вкладку, счетчик уменьшается. Когда последний закрывает вкладку (счетчик = 0), сервер выгружает документ (вызов деструктора).

3.2 Пример использования

```
1 #include <memory>
2 #include <iostream>
3
4 void observe(std::shared_ptr<int> p) {
5     std::cout << "Count inside function: " << p.use_count() << "\n";
6 }
7
8 int main() {
9     // Правильное создание (одна аллокация памяти)
10    std::shared_ptr<int> p1 = std::make_shared<int>(100);
11    std::cout << "Initial count: " << p1.use_count() << "\n"; // Выведет 1
12
13    {
14        // Копирование разрешено!
15        std::shared_ptr<int> p2 = p1;
16        std::cout << "Count after copy: " << p1.use_count() << "\n"; // Выведет 2
17
18        // При передаче по значению счетчик временно станет 3
19        observe(p1);
20
21    } // p2 уничтожается, счетчик снова 1
22
23    std::cout << "Final count: " << p1.use_count() << "\n"; // Выведет 1
24    return 0;
25 } // p1 уничтожается, счетчик равен 0, память освобождается
```

4 std::weak_ptr (Разрушитель циклов)

4.1 Теория: Зачем нужен?

Главная уязвимость `shared_ptr` — **циклические ссылки (Cyclic References)**. Если объект А хранит `shared_ptr` на Б, а Б хранит `shared_ptr` на А, они будут держать друг друга в памяти вечно. Их счетчики никогда не опустятся ниже 1, даже если вся остальная программа про них забудет.

Решение: `weak_ptr`. Это "умный указатель-наблюдатель".

- Он создается из `shared_ptr`, но **не увеличивает** счетчик сильных ссылок (`shared_count`). Он увеличивает только `weak_count` в Control Block.
- У него нет оператора `->` и `*`. Чтобы работать с объектом, нужно вызвать метод `lock()`, который превратит его во временный `shared_ptr` (если объект еще жив).
- **Важно:** Память из-под объекта освобождается, когда `shared_count == 0`. Но сам **Control Block** уничтожается только тогда, когда и `shared_count == 0`, и `weak_count == 0`.

Суть на пальцах (Жизненная аналогия)

`weak_ptr` — это ты, стоящий на улице и смотрящий в окно ресторана. Твое присутствие никак не влияет на то, работает ресторан или нет. Если ресторан закрылся, ты просто увидишь темные окна.

Но если ты хочешь съесть стейк, ты не можешь сделать это через окно. Тебе нужно зайти внутрь — это вызов метода `lock()`. Ты получаешь статус клиента (`shared_ptr`), ешь, а потом выходишь.

4.2 Пример использования (разрыв цикла)

```
1 #include <memory>
2 #include <iostream>
3
4 struct Node {
5     // Совместное владение здесь создаст утечку при цикле!
6     // Правильное решение: слабая ссылка
7     std::weak_ptr<Node> neighbor;
8
9     ~Node() { std::cout << "Node destroyed\n"; }
10 };
11
12 int main() {
13     auto nodeA = std::make_shared<Node>();
14     auto nodeB = std::make_shared<Node>();
15
16     // Создаем цикл
17     nodeA->neighbor = nodeB;
18     nodeB->neighbor = nodeA;
19
20     // Проверяем, жив ли сосед
21     if (std::shared_ptr<Node> locked_B = nodeA->neighbor.lock()) {
22         std::cout << "Neighbor is alive! Doing work...\n";
23     } else {
24         std::cout << "Neighbor is dead.\n";
25     }
```

```
26
27 // При выходе из main, nodeA и nodeB корректно удалятся,
28 // так как слабая ссылка не держит счетчик ссылок!
29 return 0;
30 }
```